

Agri Business Management System

Technology Stack Overview

A crop procurement and agricultural trading management system covering farmers, purchases, transport, warehouse inventory, buyers, sales, payments, and reporting — with role-based access (OWNER/STAFF) and a full audit log.

Tech Stack Summary

Layer	Technologies
Frontend	React, Vite, TypeScript, Tailwind CSS, React Router, Axios
Backend	Node.js, Express, TypeScript, Zod, Prisma, JWT, bcrypt, Helmet, express-rate-limit, Morgan
Database	PostgreSQL

Frontend

React + Vite single-page application, written in TypeScript.

- React — UI library
- Vite — build tool / dev server
- TypeScript — static typing
- Tailwind CSS — utility-first styling
- React Router — client-side routing
- Axios — HTTP client for calling the backend API

Structure: **pages/** (one folder per feature area), **components/** (shared UI — Button, TextField, SelectField, ConfirmDialog, ...), **services/** (Axios calls to the backend API), **utils/** (shared formatting for currency, weight, dates).

Backend

Node.js + Express REST API, written in TypeScript.

- Node.js + Express — HTTP server / API framework
- TypeScript — static typing
- Prisma — ORM and database migrations
- Zod — request validation schemas
- JWT (pinned algorithm) — authentication
- bcrypt — password hashing
- Helmet — security headers
- express-rate-limit — rate limiting (login + general API)
- Morgan — HTTP request logging

Structure: **routes/** (one file per resource, wires auth + validation + controller) → **controllers/** (thin HTTP layer, maps service errors to status codes) → **services/** (all business logic and Prisma queries), plus **validators/** (Zod schemas) and **middleware/** (authenticate, authorize, rate limiting, error handling).

Database

PostgreSQL, accessed through Prisma. Schema, migrations, and a development-only seed script live in **backend/prisma/** (schema.prisma, migrations/, seed.ts — seeding is blocked automatically in production).

Security Notes

- Every API route requires a valid JWT except POST /api/auth/login.
- Role-based access (OWNER/STAFF) enforced on every route; OWNER required for financially sensitive areas (dashboard, reports, audit log, price adjustments, inventory adjustments, staff/payroll data).
- Passwords hashed with bcrypt — no endpoint ever returns a password hash.
- JWT_SECRET has no usable fallback in production.
- Every displayed amount (purchase totals, balances, inventory, profit/loss) is recalculated server-side from source records on every request — the frontend cannot submit a trusted final figure.
- Errors are centrally handled and never leak raw database or stack trace details to the client.